

BirdFlow AI Builder — Capability Review

Answers to the 20-section architecture questionnaire:
data model, AI tool coverage, component editing,
preview/publish parity, design system, motion, media,
content safety, limits — and the decisions still needed.

BirdFlow — AI website builder

Prepared from the current codebase, with file and line citations throughout

BirdFlow AI builder — answers to the 20-section capability questionnaire

All answers are traced from the current codebase with `file:line` citations. Where a question asks "which do you want", I give a **recommendation** — those still need the product owner's sign-off and are collected in the last section.

One caveat up front: the development database is currently unreachable (tracked separately), so everything below is read from source, not reproduced live. Section 19 says explicitly which failures are code-traced rather than observed.

1. Website and page data model

There is no pages table. Everything lives in one JSONB blob.

```
builder_state(id serial PK, website_id varchar UNIQUE, state jsonb NOT NULL, revision integer NOT NULL DEFAULT 1, created_at, updated_at)
— shared/schema.ts:508-522 .
```

The state type (`shared/schema.ts:492-505`):

```
{
  pages: BuilderPage[];
  activePage: string;
  globalStyles: DesignTokens;
  stylePreset?: string;
  media?: ...;
  bookingConfig?: ...;
  checkoutConfig?: ...;
  productGridConfig?: ...;
  customComponents?: CustomComponentEntry[];
  brandGuide?: BrandGuide;
}
```

A page is only this (`shared/schema.ts:421-427`):

```
{ id: string; name: string; path: string; hidden?: boolean; components: BuilderComponentData[] }
```

Answering the specific storage questions:

Field	Where
Page name	<code>page.name</code>
Slug	<code>page.path</code>
Navigation label	Does not exist — the nav uses <code>page.name</code>
SEO title / description	Does not exist anywhere. No per-page SEO fields at all
Section order	Array order of <code>page.components</code>
Header/footer visibility	Not a page field — header and footer are <i>components inside</i> each page's <code>components</code> array (<code>shared/websiteTemplates.ts:55,189</code>)

Navigation is derived, not stored. There is no `navigation` key in the state. The nav is built from `pages`, and `page.hidden` is what excludes a page from it (`shared/schema.ts:421-427`). The `NavigationPlan` type in `shared/websitePlanSchema.ts:169-211` is an AI *planning* schema — it is never persisted.

Headers and footers are per-page, not global. Because they are ordinary components sitting in each page's array, six pages means six independent header components. Nothing keeps them in sync. This is one of the most consequential facts in this document: "change the header" is currently an N-page edit, and there is no single object to point a plan step at.

Manual page operations: create (`client/src/pages/builder.tsx:989-1012`), rename — which also regenerates the slug for non-home pages (`:1014-1034`), delete — blocked when only one page remains (`:1036-1059`). **No duplicate and no reorder.** Page array order is the displayed order (`:1275-1314`) but there is no UI to change it.

AI page operations: `add_page`, `remove_page`, `update_page` (rename/path). No reorder, no duplicate.

Size behaviour: the entire blob is sent on every save — the client autosaves the complete state after a 2 s debounce (`client/src/pages/builder.tsx:252-272`). No compression, no partial persistence, no size guard. A six-page site with rich props and custom trees is one large read-modify-write per edit.

2. Current AI tool coverage

Read tools (`buildReadTools()` , `server/aiAgentTools.ts:202-333`) — this is the Plan-mode registry:

Tool	Mutates
<code>list_pages</code>	no
<code>get_page</code>	no
<code>get_component</code>	no
<code>get_brand_guide</code>	no
<code>list_custom_components</code>	no
<code>analyze_design</code>	no
<code>run_self_check</code>	no

Write tools (`server/aiAgentTools.ts:342-473`): `add_component` , `update_component` , `remove_component` , `move_component` , `duplicate_component` , `add_page` , `remove_page` , `update_page` , `set_global_styles` , `apply_preset` , `add_section` , `create_custom_component` , `update_custom_component` , `update_brand_guide` , `set_motion` .

Non-mutating helpers: `generate_image` (`:475-519`), `propose_palettes` (`:537-558`), `propose_font_pairs` (`:560-579`), `plan_site` (`:582-621`), `analyze_reference_image` (`:623-678`), `finish` (`:680-689`).

Your table:

Operation	Exists?	Tool
Read all pages	Yes	<code>list_pages</code>
Read one page	Yes	<code>get_page</code>
Create page	Yes	<code>add_page</code>
Rename page	Yes	<code>update_page</code>
Delete page	Yes	<code>remove_page</code> (approval-gated)
Reorder pages	No	no tool, no mutation
Edit navigation	No	nothing to edit — nav is derived
Create section	Yes	<code>add_component</code> , <code>add_section</code>
Update section props	Yes	<code>update_component</code>
Update section styles	Yes	<code>update_component</code>
Reorder sections	Yes	<code>move_component</code>
Delete section	Yes	<code>remove_component</code>
Create custom component	Yes	<code>create_custom_component</code>
Update custom component	Yes	<code>update_custom_component</code>
Save component to library	Partly	only a <code>saveToLibrary</code> flag on creation (<code>shared/aiBuilderSchema.ts:268-285</code>), no standalone operation
Generate image	Yes	<code>generate_image</code>
Generate SVG	Partly	no dedicated tool — SVG is markup inside a primitive node of <code>create_custom_component</code>
Update global colours	Yes	<code>set_global_styles</code> / <code>update_brand_guide</code>
Update page colours	No	there is no page style level
Update typography	Yes	<code>set_global_styles</code> / <code>update_brand_guide</code> (global only)
Add motion	Yes	<code>set_motion</code> (sections only)
Update SEO	No	no SEO fields exist to update
Run self-check	Yes	<code>run_self_check</code>

Complete mutation union — 14 kinds (`shared/aiBuilderSchema.ts:321-336`): `add_component` , `update_component` , `remove_component` , `move_component` , `duplicate_component` , `add_page` , `remove_page` , `update_page` , `update_global_styles` , `apply_preset` , `add_section` , `add_custom_component` , `update_custom_component` , `update_brand_guide` .

Guard order in `applyWrite` (`server/aiAgentTools.ts:134-174`): large-change classifier → plan-scope guard → `validateMutation` → apply. The classifier (`server/largeChange.ts:19-59`) forces approval for any `remove_page` , `update_brand_guide` , `apply_preset` , 3+ section removals, >12 cumulative mutations, or removing >50 % of a page's sections. `NEVER_AUTONOMOUS_ACTIONS` is narrower — only `update_brand_guide` and `apply_preset` (`shared/assistantPlan.ts:102-110`).

3. Premade-component editing system

Yes — editability is registry-driven and the Properties panel is fully schema-driven.

```
type FieldType = 'text' | 'textarea' | 'color' | 'select' | 'image'
  | 'image-array' | 'items' | 'range' | 'styled-text' | 'boolean';
// shared/componentRegistry.ts:6

interface FieldDefinition {
  key; label; type: FieldType; group: 'content' | 'style';
  options?; placeholder?; min?; max?; step?; unit?;
}
// :330-341

interface ComponentDefinition {
  type; name; icon; defaultProps; defaultStyles; fields: FieldDefinition[];
}
// :562-569
```

Note the naming: there is no separate `editableFields` — **fields** *is* the editable schema.

Against your control list: text **Yes**, rich text **Partly** (`styled-text` , a limited styled string — not a full rich-text model), image **Yes** (`image` and `image-array`), link **No** (plain `text` fields such as `buttonLink`), colour **Yes**, font **No** (no font field on any premade component — fonts are global only), spacing **Partly** (a `text` field for `padding` , free-form CSS), alignment **Partly** (a `select`), repeater **Yes** (`items`), toggle **Yes** (`boolean`), select **Yes**, number/slider **Yes** (`range`).

The Properties panel (`client/src/components/builder/PropertiesPanel.tsx:68-99, 170+, 735-740`) looks up `componentRegistry[component.type]` , splits `fields` by `group` , and routes content fields to `props` and style fields to `styles` . So **props and styles are cleanly separated**.

Inline canvas editing does *not* use the schema. `InlineTextEditor` takes an arbitrary `field` prop and writes `{props:[field]: value}` (`client/src/components/builder/InlineTextEditor.tsx:4-12,117-128`). It works, but it is schema-independent, so the two editing paths can disagree about what is editable.

Nested items are not individually selectable in premade components. `items` is declared as a single field with a bespoke collection renderer (`PropertiesPanel.tsx:450`); `FieldDefinition` has no per-item field schema. You edit the list in the panel, not by clicking one card on the canvas. Nested selection exists only for custom primitive trees (see §4).

Repeatable content exists on `features` (3 items, `:717-737`), `testimonials` (3, `:743-767`), `pricing` (3 plans, `:955-987`), `faq` (4, `:990-1017`).

Critically: there is no responsive data on premade components at all. `BuilderComponentData` is `{id, type, props, styles}` (`:555-560`) — no `tabletStyles` / `mobileStyles` . Only custom primitive nodes are responsive. Premade sections rely on whatever the renderer hardcodes.

One simple entry (`hero` , `shared/componentRegistry.ts:582-623`):

```

hero: {
  type: 'hero', name: 'Hero Section', icon: 'layout',
  defaultProps: {
    styledTitle: { text: 'Byg din hjemmeside på få minutter' },
    styledSubtitle: { text: 'Smukke sider uden kode' },
    styledDescription: { text: 'Skab professionelle hjemmesider ...' },
    buttonText: 'Kom i gang gratis', buttonLink: '#',
    secondaryButtonText: 'Se hvordan det virker', secondaryButtonLink: '#',
    layout: 'centered', alignment: 'center',
  },
  defaultStyles: {
    backgroundColor: '#1a1a2e', backgroundOpacity: 100, textColor: '#ffffff',
    padding: SECTION_SPACING.hero, buttonColor: '#4f46e5', buttonHoverColor: '#4338ca',
  },
  fields: [
    { key: 'styledTitle', label: 'Title', type: 'styled-text', group: 'content' },
    { key: 'styledSubtitle', label: 'Subtitle', type: 'styled-text', group: 'content' },
    { key: 'styledDescription', label: 'Description', type: 'styled-text', group: 'content' },
    { key: 'buttonText', label: 'Button Text', type: 'text', group: 'content' },
    { key: 'buttonLink', label: 'Button Link', type: 'text', group: 'content' },
    { key: 'secondaryButtonText', label: 'Secondary Button Text', type: 'text', group: 'content' },
    { key: 'secondaryButtonLink', label: 'Secondary Button URL', type: 'text', group: 'content' },
    { key: 'layout', label: 'Layout Variant', type: 'select', group: 'content',
      options: ['centered', 'split-left', 'split-right', 'minimal', 'bold', 'video-bg'] },
    { key: 'imageUrl', label: 'Image (background or split panel)', type: 'image', group: 'content' },
    { key: 'alignment', label: 'Text Alignment', type: 'select', group: 'content',
      options: ['left', 'center', 'right'] },
    { key: 'backgroundColor', label: 'Background', type: 'color', group: 'style' },
    { key: 'backgroundOpacity', label: 'Background Opacity', type: 'range', group: 'style',
      min: 0, max: 100, step: 5, unit: '%' },
    { key: 'buttonColor', label: 'Button Color', type: 'color', group: 'style' },
    { key: 'buttonHoverColor', label: 'Button Hover Color', type: 'color', group: 'style' },
    { key: 'textColor', label: 'Text Color', type: 'color', group: 'style' },
    { key: 'padding', label: 'Padding', type: 'text', group: 'style', placeholder: '80px 24px' },
  ],
}

```

One repeating entry (`faq`, :990-1019):

```

faq: {
  type: 'faq', name: 'FAQ Accordion', icon: 'help-circle',
  defaultProps: {
    styledTitle: { text: 'Ofte stillede spørgsmål' },
    styledSubtitle: { text: 'Har du spørgsmål? Vi har svarene.' },
    title: 'Ofte stillede spørgsmål',
    subtitle: 'Har du spørgsmål? Vi har svarene.',
    items: [
      { id: '1', title: 'Hvordan kommer jeg i gang?', description: '...' },
      { id: '2', title: 'Er der en gratis prøveperiode?', description: '...' },
      { id: '3', title: 'Kan jeg opsige når som helst?', description: '...' },
      { id: '4', title: 'Kan jeg bruge mit eget domæne?', description: '...' },
    ],
  },
  defaultStyles: { backgroundColor: '#ffffff', textColor: '#1a1a1a',
    padding: SECTION_SPACING.comfortable },
  fields: [
    { key: 'styledTitle', label: 'Title', type: 'styled-text', group: 'content' },
    { key: 'styledSubtitle', label: 'Subtitle', type: 'styled-text', group: 'content' },
    { key: 'items', label: 'FAQ Items', type: 'items', group: 'content' },
    { key: 'backgroundColor', label: 'Background', type: 'color', group: 'style' },
    { key: 'textColor', label: 'Text Color', type: 'color', group: 'style' },
    { key: 'padding', label: 'Padding', type: 'text', group: 'style', placeholder: '60px 24px' },
  ],
}

```

Note `faq` carries both `styledTitle` and a legacy `title` — duplicated content keys like this are a real hazard for an AI that edits props by name.

4. Custom-component editing

```
PRIMITIVE_STYLE_KEYS = ['display','flexDirection','flexWrap','justifyContent','alignItems','gap',
  'gridTemplateColumns','alignSelf','flex','order','width','height','maxWidth','minWidth','minHeight',
  'aspectRatio','padding','margin','backgroundColor','backgroundImage','backgroundSize',
  'backgroundPosition','color','border','borderRadius','boxShadow','opacity','overflow','fontFamily',
  'fontSize','fontWeight','lineHeight','letterSpacing','textAlign','textTransform','textDecoration',
  'objectFit','transform','transition'] as const; // shared/customComponents.ts:28-50

type PrimitiveNode = {
  id: string; type: 'box'|'text'|'image'|'button'|'svg'; name?: string;
  styles?: PrimitiveStyles; tabletStyles?: PrimitiveStyles; mobileStyles?: PrimitiveStyles;
  text?: string; tag?: PrimitiveTextTag;
  src?: string; alt?: string; mediaId?: string;
  label?: string; href?: string; variant?: 'primary'|'secondary'|'outline'|'ghost'|'link';
  svg?: string; children?: PrimitiveNode[];
}; // :60-91

type CustomComponentEntry = {
  id: string; name: string; source: BuilderComponentData;
  createdAt: string; updatedAt?: string;
}; // :98-105
```

`editableSchema` / `EditableFieldDefinition` **does not exist**. The AI schema carries only the recursive primitive tree (`shared/aiBuilderSchema.ts:223-285`). This is the single biggest gap in the questionnaire and your instinct about `{ tree, editableSchema }` is correct.

What the customer can actually do today with an AI-created custom component — better than you might expect:

Capability	Works?	Where
Select it as a section	Yes	<code>client/src/pages/builder.tsx:860-869</code>
Select a nested node	Yes	layer tree <code>CustomComponentEditor.tsx:254-273</code> ; canvas click <code>CustomComponentRenderer.tsx:189-218</code>
Edit text inline	Yes	double-click contentEditable, <code>CustomComponentRenderer.tsx:100-171</code>
Replace images	Yes	upload or URL, <code>CustomComponentEditor.tsx:449-491</code>
Button label + link + variant	Yes	<code>:495-520</code>
Edit colours	Yes	per-node style controls <code>:277-341</code>
Edit spacing	Yes	same
Edit typography	Yes	same, plus text-tag selector <code>:433-445</code>
Responsive (device tabs)	Yes	<code>:187-198</code> , base/tablet/mobile
Add / remove repeated items	Partly	only as raw nodes — duplicate/delete a child (<code>:358-407</code>). There is no repeater concept, so "add another card" is a tree operation, not a content operation

So the missing piece is not raw editability — it is **semantics**. The customer can edit any node but nothing tells them which node is "the headline" or "the price"; and there is no list abstraction.

AI updating in place: yes — `update_custom_component` with `pageId` , `componentId` , optional name/tree/styles (`shared/aiBuilderSchema.ts:278-285`). Note it targets an *inserted instance*, not a library entry.

Instances are copies. Inserting deep-clones `source` and reassigns every id in the tree (`shared/customComponents.ts:324-346` , `client/src/pages/builder.tsx:873-885`). No definition reference, no overrides. Changing or deleting the library entry has zero effect on inserted instances (`builder.tsx:924-932`).

Safety limits worth knowing: sanitization on save/render/publish strips unknown nodes and style keys and caps the tree at **400 nodes / depth 24** (`shared/customComponents.ts:632-682, 697-708`).

5. Component instances versus definitions

Today BirdFlow is **Option A (copied instance)**, unambiguously. There is no override layer and no link — the four concepts you list collapse into two: a library snapshot and an independent copy.

Recommendation: stay on A for now, and design toward C (hybrid) rather than B.

Reasoning: the win from B is "restyle every instance at once", but on this codebase that win is smaller than it looks, because there is no token layer (§9) — a linked instance would still carry literal hex, so a palette change would not propagate anyway. Tokens are the prerequisite. Do tokens first, then a linked instance becomes cheap and genuinely useful. Doing B before tokens buys complexity without the payoff.

The one change I would make now: give library entries stable identity and versioning metadata (§18) so that a later move to linked instances is a migration, not a rewrite.

6. Preview and publish pipeline

This is the area with the most risk, and it deserves a blunt answer: **preview and publish are two separate renderer implementations of the same data.** They agree today by discipline, not by construction.

- **Preview** uses `client/src/components/builder/ComponentRenderer.tsx` directly (`client/src/pages/builder.tsx:1405-1407`). Custom components go to `CustomComponentRenderer.tsx`, whose recursive `NodeRenderer` (`:189-245`) resolves styles with the **shared** `resolvePrimitiveStyles(node, deviceMode)` from `shared/customComponents.ts` and applies them **inline**.
- **Publish** is `generateNextJsProject` (`server/publisher/generator.ts:182`). It sanitizes the state (`:222-224`) and emits a standalone Next.js project *including a generated copy of a ComponentRenderer* (`generateComponentRenderer`, `server/publisher/templates.ts:5065+`).

So the answer to "which approach": **serialized JSON embedded in generated Next.js source, rendered by a second, generated runtime renderer.** Not generated per-component React markup, not a shared renderer.

Does publishing serialize the primitive tree? Yes — `CustomComponentSection` reads `props.customTree` (`templates.ts:5044-5046`).

Same style resolver? No. The builder uses `resolvePrimitiveStyles`; the publisher has its own emitters — `customNodeBaseStyles` (`templates.ts:4926-4942`), `customStyleBlock` (`:4958-4969`), `collectCustomCss` (`:4971-4980`). Same documented cascade, two implementations.

Responsive: correctly emitted, as CSS rather than inline. Every node gets a `.pn-<id>` class (`templates.ts:4889-4891`), and base/tablet/mobile rules are wrapped in `@media (max-width:1024px)` / `@media (max-width:640px)` (`:4971-5060`). Breakpoints match the builder.

SVG: published correctly — sanitized in the generator (`generator.ts:222-224`) and injected via `fitCustomSvg` + `dangerouslySetInnerHTML` (`templates.ts:4982-4989, 5036-5038`).

Motion: published for *sections* — the preview `AnimatedWrapper` (`ComponentRenderer.tsx:106-170`) is duplicated as a template-side `AnimatedWrapper` (`templates.ts:2716-2855`). But primitive nodes have **no animation key at all** — only `transform` and `transition` (`shared/customComponents.ts:28-46`). So motion inside a custom component is impossible except SMIL inside an SVG.

Fonts — a real parity bug. The preview injects a Google Fonts `<link>` for *any* family name (`client/src/pages/onboarding.tsx:132-142`, `ReadOnlySitePreview.tsx:43-50`). The published site emits a **fixed, hardcoded family list** (`templates.ts:6594-6619`). An arbitrary Google Font therefore renders correctly in preview and silently falls back to a system font after publishing.

Known preview/publish divergences found:

1. Font loading, as above — silent, and exactly the class of bug you called unacceptable.
2. `product-detail` is handled by the preview switch (`ComponentRenderer.tsx:4069-4135`) but **not** by the publisher switch (`templates.ts:5065-5128`).
3. The published `ContainerSection` renders an **empty div** with a comment that children are the parent's job (`templates.ts:4841-4859`), while the preview container receives `allComponents` and renders them (`ComponentRenderer.tsx:4129-4130`).
4. `prefers-reduced-motion` is respected in neither renderer. There is a generic rule in `client/src/index.css:251` (builder chrome only); the published globals have keyframes and transitions with no reduced-motion rule.
5. Neither renderer supports `:hover` states for custom nodes — the publisher flattens style keys into declarations, so hover only "works" if a `transition` happens to be set.

Main files/functions:

Concern	Preview	Published
Custom-component rendering	<code>CustomComponentRenderer.tsx</code> <code>NodeRenderer</code> <code>:189-245</code>	<code>templates.ts</code> <code>CustomNode</code> / <code>CustomComponentSection</code> <code>:5009-5061</code>
Responsive styles	<code>shared</code> <code>resolvePrimitiveStyles</code> (<code>shared/customComponents.ts:542-560</code>), inline	<code>customNodeBaseStyles</code> / <code>customStyleBlock</code> / <code>collectCustomCss</code> (<code>templates.ts:4926-4980</code>)
Motion	<code>AnimatedWrapper</code> (<code>ComponentRenderer.tsx:106-170</code>)	<code>AnimatedWrapper</code> (<code>templates.ts:2716-2855</code>)
Fonts	dynamic <code><link></code> injection	fixed family list (<code>templates.ts:6594-6619</code>)

7. Global design system and per-section styling

```
type BrandGuide = {
  colors: { primary, secondary, accent, background, surface, text }; // :111-118
  typography: { headingFont, bodyFont, scale: 'modern'|'editorial'|'classic'|'bold' }; // :122-126
  spacing: 'tight' | 'normal' | 'airy';
  radius: 'none' | 'soft' | 'rounded'; // → 0 / 8 / 16 px, :267-277
  shadow: 'none' | 'subtle' | 'elevated';
  motion: 'none' | 'subtle' | 'expressive';
  motionSpeed: ...;
  toneOfVoice?: string;
  tone?: { principles, doWords, avoidWords, sampleHeadings }; // :187-229
}; // shared/customComponents.ts:198-212
```

Against your list: brand colours **Yes** (flat six), semantic roles **Partly** (no `heading` / `body` / `border` — one `text` colour does all three), heading font **Yes**, body font **Yes**, type scale **Partly** (qualitative keyword, no numbers), spacing scale **Partly** (keyword), border radius **Yes** (keyword → px), shadows **Partly** (keyword, no values), button style **No** (in component styles only), container width **No**, motion preference **Yes**.

Style levels that actually exist:

1. Global — `globalStyles` / brand guide
2. Component (= section) — `component.styles`
3. Primitive node — `styles` → `tabletStyles` → `mobileStyles`

There is no page level and no separate section-vs-component level. The cascade is:

```
primitive mobileStyles → primitive tabletStyles → primitive styles
→ component styles → global styles/theme → component defaults
```

Resolvers: `shared/customComponents.ts:553-559` (narrower breakpoint wins by `Object.assign`) and `shared/rendering/componentStyles.ts:9-27` (component values spread over theme, so local wins).

The cascade is clear and one-directional, which is good. What is missing is a **page level** and a **token indirection layer** — see §9.

8. Typography

- **Globally:** yes, a heading/body font pair in `GlobalStyles` (`shared/aiBuilderSchema.ts:101-115`).
- **Per section:** **No** — no premade component has a `font` field. A section inherits the global pair.
- **Per text node:** **Yes** inside custom components — `fontFamily`, `fontSize`, `fontWeight`, `lineHeight`, `letterSpacing`, `textTransform`, `textAlign`, `textDecoration`, `maxWidth` (`shared/customComponents.ts:28-46`), with tablet/mobile overrides (`:64-70`).
- **Preview loading:** dynamic `<link rel=stylesheet>` injection for any family name (`client/src/pages/onboarding.tsx:132-142`).
- **Publish loading:** a fixed family list only (`server/publisher/templates.ts:6594-6619`).
- **Allowlist:** ~50 presets exist for the *UI pickers* (`shared/componentRegistry.ts:166-232` , used by `GlobalStylesPanel.tsx:152-160`), but nothing enforces them on the AI or on stored values.
- **Can the AI install a Google Font automatically?** It can *store* one. It cannot make the published site load it.
- **Type scale:** individual sizes; the brand `scale` is a qualitative keyword, not numbers.

AI control: family **Yes**, weight **Yes**, size **Yes**, line-height **Yes**, letter-spacing **Yes**, transform **Yes**, max width **Yes**, responsive **Yes** — **but all of that only inside custom components**. For premade sections the AI can only change the global pair.

Recommendation on fonts: approved allowlist, not any Google Font. Concretely: keep the ~50-font preset list as the single source of truth, drive both the preview injector and the publisher's font link from it, and reject anything outside it at the schema level. Arbitrary fonts are the direct cause of the silent preview/publish divergence above, and on Danish psychology sites the aesthetic upside of font #51 is near zero. If you want more range, grow the list deliberately.

9. Colour

1. **Is the brand guide the source of truth?** Partly. It is authored during onboarding and read by the AI (`get_brand_guide`) and by image generation, but it is **not** the render-time source — components carry their own values.
2. **Semantic or flat?** Flat six keys: `primary`, `secondary`, `accent`, `background`, `surface`, `text` (`shared/customComponents.ts:111-118`). No `heading`, `body`, or `border` roles.
3. **Can a component reference `var(--brand-primary)`?** **No**. Style values are raw CSS strings (`shared/aiBuilderSchema.ts:58-86`); defaults are literal hex such as `#475569` (`shared/customComponents.ts:463, 515-524`).
4. **So each component stores literal hex.** Yes. This is the root cause of "the AI changed the palette but the site still looks the same" — and of any future palette work being an N-section rewrite.
5. **Should AI changes to global colour roles update the whole site?** **Yes** — and this is exactly why tokens should be introduced. With tokens it is one write; without them it is dozens of writes that can partially fail.
6. **Local overrides allowed?** Yes, keep them — but they should be visibly *overrides* of a role, not anonymous hex.
7. **Whole-palette change requires approval?** Already true: `update_brand_guide` and `apply_preset` are in `NEVER_AUTONOMOUS_ACTIONS` (`shared/assistantPlan.ts:102-110`) and flagged by the large-change classifier (`server/largeChange.ts:19-59`). Keep it.
8. **Generate and compare several palettes? Already exists** — `propose_palettes` returns 4 palettes as clickable cards (`server/aiAgentTools.ts:537-558`), and `propose_font_pairs` does the same for type (`:560-579`).

I agree with your assessment: semantic tokens are the highest-leverage structural change in this whole document, and several other items (linked instances, whole-site restyle, brand consistency in self-review) are blocked behind it.

10. Motion system

Current schema (`shared/aiBuilderSchema.ts:87-92`):

```
animationType: 'none' | 'fade-in' | 'slide-up' | 'slide-down' | 'slide-left' | 'slide-right'
  | 'zoom-in' | 'zoom-out' | 'bounce' | 'flip';
animationTrigger: 'load' | 'scroll';
animationDuration: string; // unconstrained in the schema
animationDelay: string; // unconstrained in the schema
```

The `set_motion` tool narrows it further: `duration` $\in \{0.3s, 0.5s, 0.8s, 1.2s\}$, `delay` $\in \{0s, 0.1s, 0.3s, 0.5s\}$ (`server/aiAgentTools.ts:442-471`).

Against your proposed schema: `animationType` **Yes**, `trigger` **Yes** (load/scroll only), `duration` **Yes**, `delay` **Yes**, `easing` **No**, `distance` **No**, `direction` **Partly** (baked into the type name), `stagger` **No**, `repeat` **No**.

- **Attached to:** sections/components only. **Not** primitive nodes.
- **Preview:** **Yes** `AnimatedWrapper` with `load` and `IntersectionObserver` `scroll` (`ComponentRenderer.tsx:106-170`).
- **Publish:** **Yes** duplicated generated wrapper (`templates.ts:2716-2855`).
- **Scroll reveal** **Yes**. **Entrance** **Yes**. **Hover motion** **No**. **Looping decorative** **Partly** — only a built-in marquee (`templates.ts:4500-4506`) and SMIL inside sanitized SVG.
- **Stagger:** not a field. The tool *description* tells the model to fake it with increasing delays on consecutive sections (`:445-446`). Works, but it is prompt convention rather than a guarantee, and it cannot stagger children within one section.
- **prefers-reduced-motion:** **No** not respected in either renderer.
- **Unsupported values rejected?** Partly — Zod rejects bad `animationType` / `trigger` enums, but `duration` and `delay` are free strings in `ComponentStylesSchema`; only the tool path is narrow.

The reduced-motion gap is the one I would treat as non-negotiable before making motion more expressive: for a psychology practice's audience, unstoppable movement is an accessibility problem, not a polish item.

11. SVG illustration capability

- **Raw markup, not a structured model.** `PrimitiveNode.svg?: string` (`shared/customComponents.ts:60-90`), created via `create_custom_component` (`server/aiAgentTools.ts:413-421`).
- **Stored inline in the node**, not as an asset in the media library. So an SVG cannot be reused across pages except by copying the node, and it is not listed anywhere as an asset.
- **Brand tokens?** **No** `fill` / `stroke` are literal attribute values; no substitution layer.
- **Editable later?** Only by regenerating the node or hand-editing markup. There is **no UI** to change main colours, stroke width, icon details, or text inside the SVG. The primitive editor treats the SVG node as an opaque blob.

- **Motion on the whole SVG?** Only via the section-level `set_motion` (the SVG rides along with its section). **Individual paths:** only via SMIL inside the markup — `animate`, `animateTransform`, `animateMotion` are explicitly allowlisted (`shared/svgSanitizer.ts:34-90`) and the tool description advertises this.
- **Sanitizer:** max 300 000 chars (`shared/svgSanitizer.ts:16`); blocks `script`, `style`, `foreignObject`, `iframe`, `object`, `embed`, `audio`, `video`, `math`, `template`, `feImage`, `a` (:19-32); `href` restricted to internal `#id` or embedded raster data URIs (:92-93); strips event handlers, unsafe CSS and URLs, and malformed markup (:138-220).
- Tree caps apply too: 400 nodes, depth 24 (`shared/customComponents.ts:697-708`).

Your recommendation matches what is already built, with two gaps: colours are not tokenised, and 300 KB of inline SVG per node is very generous — a 300 KB illustration embedded in the state blob is a performance problem long before it is a security one. I would tighten the practical cap substantially (single-digit KB for icons, tens of KB for illustrations) and add a node count/path budget.

12. Images and media

- **Model:** OpenAI `gpt-image-1` (`server/aiImages.ts:86,150`).
- **Storage:** Replit object storage, optimised to WebP with sharp, then registered in the media library (`server/aiImages.ts:85-121`). Not R2, not Supabase Storage.
- **Breakpoint crops / mobile variants:** **No** neither exists. One rendition per image.
- **Users can replace generated images:** **Yes** upload or URL (`CustomComponentEditor.tsx:449-491`), and `image` fields on premade components.
- **Alt text stored separately:** **Yes** on the media asset as a truncated description (`server/aiImages.ts:109-119`), and also on the node (`PrimitiveNode.alt`, `shared/customComponents.ts:77-80`).
- **Brand guide reaches image generation:** **Yes** imagery style, primary/accent/background colours, imagery notes and keywords (`server/aiImages.ts:43-70`).
- **Reuse across pages:** **Yes** a generated URL can be used anywhere; reusing the exact same description is free (tool description, `server/aiAgentTools.ts:475-519`).
- **Inspect existing images before generating:** **Yes** `/objects/...` images can be read and resized for vision (`server/aiImages.ts:187-204`), and `analyze_reference_image` exists.
- **Three-image cap:** `MAX_AI_IMAGES_PER_REQUEST = 3` (`server/aiImages.ts:29`) plus `MAX_IMAGES_PER_BUILD` (`server/buildOrchestrator.ts:30`, `server/planAgent.ts:19`), with a unique-description cache so repeats are free.

On the cap: agreed, three is too few for a multi-page build and should not be removed blindly. The right move is the one you describe — make the *planner* allocate the budget explicitly. A plan for a five-page site should state which images it intends to generate and where, so the budget becomes a visible design decision the customer approves rather than an invisible ceiling the build hits at step 9. My suggested shape: keep a per-build cap, but scale it with the number of pages the plan actually touches, and have the plan list its image intentions as named line items.

13. Content and copywriting

- **Language:** `websites.language` column, default `da`, allowed `da/en` (`shared/schema.ts:177-187`, `shared/siteLanguage.ts:17-29`), with an explicit copy-prompt instruction (:63-67).
- **Tone:** `BrandGuide.toneOfVoice?: string` plus an optional structured `tone` (`principles`, `doWords`, `avoidWords`, `sampleHeadings`) at `shared/customComponents.ts:187-229`. The structured version is good and underused.

What the builder AI actually receives — this is a significant finding. The builder's system prompt contains generic rules only (`server/aiAgent.ts:84-111`). The user-side context is `buildStateSummary`: page list, active page, custom component names, brand context (:119-139). The planning prompt hardcodes "a real Danish website for a psychology practice" (`server/planAgent.ts:135-140`) and adds pages, active page, library, brand (:158-179).

Context	Reaches the builder/planner?
Business description	No (onboarding agent only, <code>server/onboardingAgent.ts:283-310</code>)
Audience	No no field anywhere
Services	Partly only if embedded in free-text goals during onboarding
Location	No no field anywhere
Conversion goal	Partly <code>websiteArchitect.ts:237-249</code> asks for it at plan time; not persisted for later edits
Tone	Yes via brand guide
Existing page copy	Partly only on demand via <code>get_page</code> — not in the initial context

So the builder is being asked to write copy for a psychology practice while knowing the brand's colours and tone but **not what the practice does, for whom, or where**. That alone explains generic output far better than any prompt-quality issue.

Answering your policy questions with recommendations:

- **Rewrite the whole site?** Only inside an approved plan that names the pages. Never as a side effect.
- **Preserve factual claims unless asked? Yes — mandatory.** Names, titles, credentials, prices, opening hours, addresses, phone numbers, CVR, and therapy modalities must be treated as immutable facts that may be re-worded but not invented or altered.
- **Prohibit inventing testimonials / qualifications / statistics / prices / treatment results? Yes — hard prohibition, enforced server-side, not by prompt.** `server/copyRules.ts` already establishes the pattern of deterministic Danish copy checks; this belongs there. For a Danish psychology practice, a fabricated authorisation ("autoriseret psykolog"), a fabricated testimonial, or a claimed success rate is a regulatory and professional-ethics problem for your customer, not a quality nit. I would make a plan step that would write into a testimonial or credentials section fail closed unless the customer supplied the content.
- **SEO-aware content?** Yes in the writing (headings, descriptive titles, local terms) — but note there are currently **no SEO fields to write into** (\$1). Meta titles and descriptions cannot be created because they do not exist in the data model. That needs to be added before it can be asked for.
- **One copy goal per section in the plan?** Yes. It makes plan steps reviewable in the customer's own language and gives the build a checkable target.
- **Image alt text?** Yes — already stored separately, so this is cheap and should be required.

14. Whole-site editing behaviour

Your recommendation matches my judgement, and I would adopt it as written, with one structural caveat that determines whether it is enforceable.

Area	Rule	Enforceable today?
Ordinary content/design edits	Approved via plan scope	Yes <code>server/planScope.ts</code> already scopes steps per page
Navigation restructuring	Explicitly listed	Partly <code>nav</code> is derived from pages and <code>hidden</code> ; "restructure navigation" = rename/reorder/hide pages, and <code>reorder</code> does not exist
Page deletion	Destructive confirmation	Yes classifier already forces approval
Brand-guide overwrite	Destructive confirmation	Yes in <code>NEVER_AUTONOMOUS_ACTIONS</code>
Booking/form logic	Styling yes, functional bindings no	Partly bindings live in <code>bookingConfig</code> / <code>checkoutConfig</code> / <code>productGridConfig</code> at state level with no AI tool to touch them — so they are safe by absence, not by rule
Legal pages	Never rewrite automatically	No not enforceable — nothing marks a page as legal
Hidden pages	—	<code>page.hidden</code> exists; the comment notes hidden pages are still published
Unpublished drafts	—	no draft concept exists
Page slugs	Should require explicit listing (changing a live slug breaks inbound links and SEO)	Partly <code>update_page</code> can change <code>path</code> freely today
SEO metadata	—	does not exist

The caveat: four of these rules ("never touch legal pages", "don't rewrite booking sections", "don't change published slugs", "leave drafts alone") all require the system to know **what kind of page this is**. Today a page is a name, a path and an array. Without a page-role concept — `legal`, `booking`, `landing`, `draft` — these protections can only be prompt suggestions, and prompt suggestions are exactly what fails on the big requests you are trying to enable.

I would add a small `role` field to the page type as a prerequisite for the whole-site work.

15. Functional components

Existing types with behaviour (`shared/componentRegistry.ts`):

Your list	Exists	Binding
Booking widget	Yes <code>booking</code> :861-883	<code>bookingConfig</code> (<code>shared/schema.ts:440-458</code>)
Contact form	Yes <code>contact-form</code> :1051-1080	no registry-level backend config
Newsletter form	Yes <code>newsletter</code> :1150-1191	no registry-level backend config
Product list	Yes <code>product-grid</code> :833-858	<code>productGridConfig</code>
Checkout button	Partly no such type — checkout is <code>product-detail</code> :885-916 + <code>checkoutConfig</code>	
FAQ accordion	Yes <code>faq</code> :990-1018	none
Tabs	Yes <code>tabs</code> :1305-1336	none
Carousel	Partly <code>image-slider</code> :625-647 ; also a layout <i>variant</i> on testimonials/team :124-131, 1491-1492	
Modal	No	
Map	No	
Video	Yes <code>video-embed</code> :1083-1108	none
Calendar	Partly only inside <code>booking</code>	
Pricing calculator	No — <code>pricing</code> has a monthly/annual toggle :955-987 , not a calculator	

Also interactive: `before-after` comparison slider (:1194-1225), `header` with cart (:770-809), `marquee` .

Can custom primitive components contain behaviour? No — by design. The file states the tree is DATA, never runtime-compiled code (`shared/customComponents.ts:4-8`). Node types are exactly `box` | `text` | `image` | `button` | `svg` (:19-22), and a button has only `label` , `href` , `variant` (:81-84). A primitive button can navigate to a sanitized href and nothing else — it cannot submit a form, open a modal, or trigger booking or checkout.

Recommendation: I agree with you, strongly — visual custom components only, with functional behaviour supplied by inserting existing trusted components.

The reasoning is specific to this codebase rather than general caution. Booking and checkout are bound to website-level config objects that no AI tool can write; that is currently your hardest safety boundary and it holds precisely *because* primitives cannot execute. An AI-invented "booking calendar" made of boxes and buttons would look convincing, take real bookings nowhere, and fail silently — the worst possible failure mode for a psychology practice whose livelihood is appointments.

The productive middle ground, when you want it later: let the AI *compose* around functional components (its own layout, its own styling, its own copy, wrapping a real `booking` section), and if you ever need more interaction, add named, reviewed behaviours to the primitive schema (`accordion` , `tab-group`) — a controlled vocabulary, never arbitrary logic.

16. AI self-review

What `server/selfCheck.ts` **inspects today — five checks, all deterministic, no model:**

Check	Scope	Repairs?
<code>buttonLink</code> / <code>secondaryButtonLink</code> pointing at a nonexistent page	premade sections	Yes auto-repairs to <code>/</code> (:80-105); skips external, <code>/#</code> , <code>/objects/</code>
WCAG AA contrast (4.5:1) on explicit solid text/background	premade sections	Yes auto-repairs text to nearest readable dark slate or white (:109-128); skips gradients and images
Broken internal <code>href</code> on a custom-tree button	custom trees	Yes auto-repairs to <code>/</code> (:142-159)
Contrast on explicit custom-node <code>color</code> / <code>backgroundColor</code>	custom trees	Yes auto-repairs text colour (:161-173)
Responsive hazards — fixed widths > 640 px, 3+ column grids, display fonts ≥ 48 px with no mobile override	custom trees	Yes via <code>guardResponsive</code> (:180-186)

Against your eleven categories: architecture **No**, content quality **No**, brand consistency **No**, responsive **Yes** (custom trees only), accessibility **Partly** (contrast only — no semantics, keyboard, ARIA, alt text, focus, or touch targets), navigation and links **Partly** (button links only — not header/footer nav), functional bindings **No**, performance **No**, motion safety **No**, preview/publish parity **No**, SEO **No** (nothing to check).

Your three-level model is right, and I would adopt it, with one addition and one caution.

- **A (auto-repair safe, deterministic things)** — this is exactly what exists and it works. Keep the rule that a repair must be deterministic and explainable in one Danish sentence.

- **B (suggest, never auto-apply subjective issues)** — agreed, and this is the level that needs a model rather than rules. Keep the output as notes attached to the build so the customer sees them.
- **C (broad design revisions require approval)** — agreed, and the mechanism already exists: the large-change classifier and `NEVER_AUTONOMOUS_ACTIONS`.

Addition: preview/publish parity belongs in level A, as a hard check rather than a suggestion. Given \$6, a check that refuses to report success when a page uses a font the publisher cannot load, or a component type the publisher does not handle, would have caught two of the divergences above.

Caution learned on the last task: self-check repairs the *whole site*, but a build step is scoped to specific pages. Reverting a deterministic repair because it landed out of scope would knowingly leave a broken site, so the current behaviour is to apply it and record it in the step's notes. Keep that principle as self-review grows — repairs are reported, never silently reverted, and never silently extended.

17. Design quality and originality

Recommendation: Balanced, leaning creative — with a hard floor on conversion clarity.

- Should each site visibly differ? **Yes**. Danish psychology practices in the same city will be compared side by side; visible sameness is a commercial problem for your customer.
- Avoid reusing the same page structure? **Yes**, but structure variation should come from real content differences (which services, which audience), not randomisation. This is another reason \$13's missing business context matters — without it, the AI has nothing to vary *on*, and "be more creative" becomes arbitrary.
- Asymmetrical layouts? **Yes**.
- Overlapping elements? **Yes, sparingly** — but note the responsive guard treats large `translateX` as a blocking hazard because overlap is the most common way a page breaks out of a phone screen.
- Decorative background shapes? **Yes** — this is exactly what the SVG primitive is good at.
- SVG illustrations combined with photographs? **Yes**, and for this sector it is a strong combination: soft illustrated shapes plus one authentic photograph reads as calm and professional.
- How bold can motion be? **Restrained**. Entrance reveals and gentle staggering, nothing looping or attention-grabbing, and respect `prefers-reduced-motion` (currently ignored — \$10). An anxious visitor is a plausible primary user for these sites.
- Conversion clarity over experimentation? **Yes, always**. One obvious primary action per page — usually "book en tid". Experiment with everything except the path to booking.

18. Component-library organisation

What an entry carries today (`shared/customComponents.ts:96-105`):

```
{ id, name, source, createdAt, updatedAt? }
```

Against your recommended metadata: `id` **Yes**, `websiteId` **No** (implicit — the library lives inside that website's state blob), `name` **Yes**, `description` **No**, `category` **No**, `tags` **No**, `source: 'ai'|'user'` **No**, `thumbnail` **No**, `createdAt` **Yes**, `updatedAt` **Yes**, `version` **No**.

- **Categories today:** none for custom components. (Category fields elsewhere in the codebase are unrelated product data.)
- **AI components under "[Website name]s egne komponenter":** they already all appear in one undifferentiated list — a name and nothing else. No distinction between what the AI made and what the customer made.
- **Thumbnails:** **No** none generated.
- **Versions:** **No** none. `updatedAt` moves; nothing is retained.
- **Rename and delete:** **Yes** both (`client/src/pages/builder.tsx:908-932`).
- **Deleting a definition:** inserted instances are **unaffected** — they are independent copies (\$5). Deletion only filters the library array (`:924-932`).
- **Duplicate detection:** **No** none. Nothing stops the AI creating "Hero 2", "Hero 3", "Hero 4" across three separate requests.

Recommendation: add `description`, `category`, `tags`, `source`, `thumbnail`, and `version` before the AI starts producing components at volume. The library is currently usable because it is nearly empty; a creative AI building multi-page sites will make it unusable quickly. Thumbnails matter most — a name-only list of AI-generated components is not browsable.

19. Existing custom-component failures

Traced from code, **not reproduced live** (the development database is unreachable). Each is a concrete, located gap rather than a guess:

1. **No editable-field schema.** The customer can edit every node but nothing labels the headline, the price, or the CTA. Editing is possible; *understanding what to edit* is not.

2. **No repeater.** "Add another card" is a raw tree duplication (`CustomComponentEditor.tsx:358-407`), not a content operation.
3. **Instances are detached copies** (`shared/customComponents.ts:324-346`). The AI cannot restyle all instances of a component; the library is a snapshot store, not a definition store.
4. **Library entries have no thumbnail, category or source** — AI output lands in a flat, name-only list (§18).
5. **Arbitrary fonts silently fall back after publishing** — preview injects any family, the publisher emits a fixed list (`templates.ts:6594-6619`). This is the "works in preview, not after publish" case you specifically called unacceptable, and it exists today.
6. **product-detail is preview-only** — absent from the publisher switch (`templates.ts:5065-5128`).
7. **Published containers render an empty div** (`templates.ts:4841-4859`) while preview renders their children.
8. **No motion on primitive nodes** — only `transform` / `transition` exist (`shared/customComponents.ts:28-46`), so animation inside a custom component is impossible except SMIL in an SVG.
9. **No hover states** in either renderer for custom nodes.
0. **prefers-reduced-motion ignored** in both renderers.
1. **SVGs are inline blobs, not assets** — not reusable, not editable through any UI, and up to 300 KB each inside the state blob that is rewritten on every autosave.

What I could **not** confirm without a live database: whether AI-created components reliably reach the library (the `saveToLibrary` flag path), whether responsive styles survive a full round trip through publish, and whether spacing is faithful in published output. Those need the database restored and a real end-to-end build — which is why proving the pipeline against a real site should come early in the programme.

20. Models and limits

Concern	Value	Location
Planning model	<code>gpt-5.1</code>	<code>server/aiAgent.ts:41</code> , used by <code>planAgent.ts:212</code>
Build model	<code>gpt-5.1</code> (same shared constant)	<code>server/aiAgent.ts:41</code> , <code>buildOrchestrator.ts:488-495</code>
Copywriting model	same builder loop — no separate model	<code>server/copyRules.ts</code> is deterministic
Image model	<code>gpt-image-1</code>	<code>server/aiImages.ts:86,150</code>
Self-review	no model — fully deterministic	<code>server/selfCheck.ts:2-5,76-139</code>
Site architect	<code>gpt-4o</code>	<code>server/websiteArchitect.ts:254,367</code>
Planning completion-token limit	4096 (shared)	<code>server/aiAgent.ts:43,243</code>
Build completion-token limit	4096 (same constant)	same
Total token budget	40 000 per loop — so per build <i>step</i> , not per build	<code>server/aiAgent.ts:45,224-225,326-330</code>
Planning turn limit	8	<code>server/planAgent.ts:32-34,212</code>
Build turn limit	8 per step	<code>server/buildOrchestrator.ts:65-66,494</code>
Step retry limit	<code>MAX_STEP_ATTEMPTS = 2</code>	<code>shared/assistantPlan.ts</code> , retry at <code>buildOrchestrator.ts:376-381</code>
Plan request character limit	4000	<code>shared/assistantPlan.ts:250-252</code>
Onboarding message limit	4000 chars, 20 msgs / 10 min	<code>server/routes.ts:721-734</code>
Rate limit	10 AI runs / 10 min / user	<code>server/aiRateLimit.ts:15-17,28-43</code>
Image cap	3 unique per run / per build	<code>server/aiImages.ts:29</code> , <code>buildOrchestrator.ts:30</code>
Cost ceiling	none — tokens and rate limit are the only controls	—
Reasoning configurable	No no <code>reasoning_effort</code> or verbosity setting anywhere	only <code>max_completion_tokens</code>

Can different models be used per role? Not today — planning, building and copywriting all share one `MODEL` constant in `server/aiAgent.ts:41` . Making it a parameter is a small change and would let planning use a stronger reasoning configuration than per-step building, which is probably the single cheapest quality win available.

Is tool-result context kept across build steps? **No.** Each `runStep` reloads state and builds a fresh context (`buildOrchestrator.ts:449-469`), and `runAgentLoop` starts a new message array with only system + user (`aiAgent.ts:219-222`). Within one step, tool calls and results accumulate (`:260-312`); across steps, only a short summary of prior results carries over (`buildOrchestrator.ts:193-215`) plus the persisted site state. This is deliberate — it keeps each step bounded — but it means a step cannot learn from what an earlier step *saw*, only from what it *did*.

Each step does get the full site state in its context object (`buildOrchestrator.ts:449-464`), but the step's opening prompt serializes only page names/ids and component `id:type` (`:205-210`); full props must be fetched with `get_page` . The legacy one-shot builder, by

contrast, stringifies everything (`server/aiBuilder.ts:1034-1071`) and grows without bound.

How large can `builder_state.state` get before things slow down? There is no enforced limit and no compression, and the whole blob is read and written on every autosave. The practical ceiling is set by three things at once: JSON size on the wire, the 300 KB-per-SVG sanitizer cap, and the AI context. I cannot give a number without a real database, but the shape of the problem is clear — a rich six-page site with several SVG illustrations is a multi-hundred-KB read-modify-write every two seconds while the customer types.

The two structural facts worth carrying into any plan

Nearly every gap above traces back to one of these:

1. **Literal values instead of tokens.** Colours, fonts and spacing are baked as raw CSS into every component and node. This is why a palette change cannot cascade, why linked instances would not help yet, and why "brand consistency" cannot be checked automatically. Tokens are the prerequisite for \$5, \$7, \$9, and part of \$16.
2. **Two renderers for one data model.** Preview and publish are independent implementations that agree by discipline. Every new capability — motion, hover, fonts, new component types — must be built twice or it becomes the next silent publish bug. \$6 lists three that already exist.

Decisions still needed from you

These are the questionnaire's "which do you want" items. My recommendation is in brackets; each needs your confirmation.

1. Instance model — [stay on copied instances now, tokens first, hybrid later]
2. Fonts — [approved allowlist only, enforced in schema and publisher]
3. Colour — [move to semantic tokens; keep whole-palette changes approval-gated]
4. Copy safety — [hard server-side prohibition on invented testimonials, credentials, statistics, prices and treatment results]
5. Whole-site editing rules — [adopt your table as written, and add a page `role` field so legal/booking/draft protections are actually enforceable]
6. Functional components — [visual custom components only; insert trusted functional components for behaviour]
7. Self-review — [adopt the three-level model, and add preview/publish parity to level A]
8. Design freedom — [balanced leaning creative; restrained motion; conversion clarity always wins]
9. Image budget — [keep a cap, scale it with the pages a plan touches, and make the planner list its image intentions]
10. Models — [make the model a per-role parameter rather than one shared constant]

On the proposed programme

I agree it should not be one task, and I agree with the sequencing instinct. Two adjustments based on what the code actually shows:

- **A semantic token layer should come before** "typography and semantic colour tools" as a separate foundation — several later items are blocked behind it.
- **Preview/publish parity should be a standing constraint on every item**, not a single task late in the list. Three divergences exist today; adding capability to only one renderer is how the fourth gets made.

Task #93 ("Make Plan mode survive big, complicated requests") remains the right first item — everything else in the programme is planned *through* plan mode, so it has to hold up first.